

Demo - ClaudeCode

Yogesh Haribhau Kulkarni

Claude Code

Workshop Overview

What we will build: A chatbot with two interfaces: CLI and Streamlit, powered by LangChain + Groq LLM.

What we will learn:

- How to use Claude Code across the **full SDLC**: from specs to deployment
- CLAUDE.md**, custom commands, skills, sub-agents, hooks, MCP servers, plugins
- Writing code, tests, and docs: *all from the terminal*
- How Plan mode, permission system, and hooks enforce quality automatically

Duration: 90–120 minutes end-to-end.

SDLC Stages We Will Cover

Stage	Activity	Claude Code Feature
1	Project bootstrap & init	/init, CLAUDE.md
2	Requirements & specs	Custom subagent, Markdown
3	Architecture design	Plan mode, @workspace
4	Implementation	Auto-accept, @file:
5	Testing (unit + synthetic)	Custom command, MCP
6	Debugging	Debug subagent, context tools
7	Refactoring	Skills, Rules
8	Documentation	Docs subagent
9	Code review	Review subagent
10	GitHub workflow	MCP GitHub server
11	Hooks & automation	Hooks system (new!)
12	Plugins	Plugin packaging (new!)

Each stage = one demo block. Run them sequentially or jump to any stage.

Environment Setup

Installation: All Platforms

- macOS / Linux (recommended native binary):**
`curl -fsSL https://claude.ai/install.sh bash—`
- Homebrew (macOS):** `brew install --cask claude-code`
- Windows PowerShell:** `irm https://claude.ai/install.ps iex—`
- Windows CMD:** `curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd`
- NPM (legacy, deprecated since v2.1.15):** `npm install -g @anthropic-ai/claude-code`
- Migrate from npm to native binary:** `claude install`
- Verify installation:** `claude doctor`

System requirements:

- macOS 10.15+, Ubuntu 20.04+/Debian 10+, Windows 10+ (WSL or Git Bash)
- 4 GB RAM minimum, active internet connection
- No Node.js dependency with native binary!

Pre-requisites Checklist

Before starting, verify:

- Start with empty folder, switch to that directory
- Conda env installed, activated: `conda --version`
- Claude Code installed: `claude doctor`
- Node.js ≥ 18 for MCP servers: `node --version`
- Python ≥ 3.10 and pip: `python --version`
- (Optional) Groq API key: console.groq.com (free tier available)
- A GitHub Personal Access Token (for MCP GitHub stage)
- Git configured: `git config --global user.name "Your Name"`

Set environment variables (add to .bashrc/.zshrc):

```
export GROQ_API_KEY="gsk..." # or your preferred LLM
export GITHUB_TOKEN="ghp..."
```

Install Python deps we'll use:

```
pip install langchain langchain-groq streamlit
pip install pytest pytest-mock faker rich
```

Authentication and First Launch

Create project directory and launch Claude Code:

```
mkdir mychatbot && cd mychatbot
git init # Claude Code anchors to git root
claude # Launch interactive session
```

Authentication options (choose one):

```
# Option 1: Anthropic Console (API billing)
claude auth login # Opens browser to platform.claude.com

# Option 2: Claude Pro / Max subscription
claude auth login # Uses claude.ai credentials

# Option 3: Enterprise (AWS Bedrock / Google Vertex)
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
```

Manage authentication:

```
claude auth status # Check current account and plan
claude auth logout # Clear stored credentials
```

Pro Tip: Unlike OpenCode, Claude Code uses your Anthropic account or API key directly – no provider selection step needed. Verify the connection: ask “Hello, which model are you?”

Choosing the Right Model

Claude Code ships three model tiers. Use the right model for the task:

Model	Speed	Cost	Best For
claude-haiku-4-5	Fast	Cheap	Subagents, exploration
claude-sonnet-4-6	Balanced	Medium	Default: daily coding
claude-opus-4-6	Thorough	Higher	Architecture, complex

Switch model inside a session:

```
/model claude-opus-4-6

# Or launch with specific model:
claude --model claude-opus-4-6

# Or set in .claude/settings.json:
```

```
{ "model": "claude-sonnet-4-6" }
```

Practical guide:

- **Haiku**: subagents doing file exploration, simple search, fast questions
- **Sonnet**: most coding tasks, refactoring, test generation
- **Opus**: architectural design, security review, complex multi-file refactors

Project Bootstrap

Teaching Claude Code about your project with CLAUDE.md

Run /init: Generate CLAUDE.md

The first command in any new project. Scans the directory and writes Claude's instruction manual.

Make sure **Plan mode** is active first (press Shift+Tab). In-side TUI (project is still empty, .git is there though):

```
/init
```

Claude will ask clarifying questions about the project. Answer them. Once the plan looks right, press Shift+Tab to switch to normal mode and approve the file creation.

What /init generates:

```
CLAUDE.md           # Project intelligence
file
.claude/settings.json # Permissions, model
config
```

Key difference from OpenCode: In Claude Code the file is called CLAUDE.md (not AGENTS.md), and Claude also supports **nested** CLAUDE.md files in subdirectories for directory-specific instructions.

Commit immediately:

```
git add CLAUDE.md
git commit -m "chore: add CLAUDE.md"
```

CLAUDE.md: The Project Memory File

CLAUDE.md is loaded automatically at the start of *every* session. Inspect and extend it with project-specific rules. Three scopes exist:

```
~/claude/CLAUDE.md # Global: all your
projects
./CLAUDE.md        # Project: everyone
on the team
```

```
./claude/settings.local.json # Local: your
overrides (gitignore)
```

Edit the project CLAUDE.md:

```
## Project: mychatbot

## Tech Stack
- Python 3.11, LangChain 0.3+, langchain-groq
- Streamlit 1.35+, Rich (CLI output), pytest +
  pytest-mock

## Build Commands
- Install: 'pip install -e ".[dev]"'
- Tests:   'pytest tests/ -v'
- Lint:    'ruff check src/'
- Format:  'black src/'

## Conventions
- Type hints on all functions
- Docstrings (Google style) on all public functions
- No hardcoded API keys: always read from env vars
- Never catch bare except

## File Map
- src/mychatbot/chain.py : LangChain chain
  factory
- src/mychatbot/cli.py   : Rich CLI chatbot
- src/mychatbot/app.py   : Streamlit web chatbot
- tests/                 : pytest test suite
```

Auto Memory: Claude Learns As It Works

A unique Claude Code feature: Claude automatically builds memory as it works, saving learnings across sessions without you writing anything.

```
# During a session, if Claude discovers something
  useful:
# "The test suite requires GROQ_API_KEY to be set,
#  even for unit tests with mocked calls"
# Claude writes this to CLAUDE.md automatically.

# You can also instruct Claude to remember things:
"Remember that we use Black for formatting, not
  autopep8.
  Add this to CLAUDE.md."

# View what Claude has learned:
/memory

# Clear auto-generated memories:
/memory clear
```

Rules files (alternative to CLAUDE.md for team rules):

```
# Create targeted rule files:
mkdir -p .claude/rules
cat > .claude/rules/python-style.md << 'EOF'
paths: ["src/**/*.py", "tests/**/*.py"]
Apply these rules to ALL Python files:
- Use Black formatting (line length: 88)
- Prefer dataclasses over TypedDicts for mutable
  data
- Always add __all__ to module-level exports
EOF
```

Requirements & Specification

Using a custom subagent to write elaborate specs

Create a Specs Subagent

In Claude Code, subagents live in .claude/agents/. Create a dedicated **Specs** subagent:

```
mkdir -p .claude/agents

cat > .claude/agents/specs.md << 'EOF'
---
name: specs
description: >
  Technical specification writer. Produces clear,
  structured Markdown spec docs from user stories
  or
  feature descriptions. Read-only: never modifies
  source
  code. Use when writing a technical spec for any
  feature.
model: claude-opus-4-6
allowed-tools: Read, Write, Glob
---
You are a senior technical writer and software
  architect.
When asked to write a spec, produce a Markdown
  document
  covering: Overview, Goals, Non-Goals, User Stories,
  Functional Requirements, Non-Functional
  Requirements,
  Data Models, API/Interface Contracts, and Open
  Questions.
Always save output to docs/specs/<feature>.md.
EOF
```

Key Claude Code subagent features:

- Subagents run in **isolated contexts**: they don't bloat your main conversation
- allowed-tools** restricts what the subagent can do (safety boundary)
- Claude automatically selects a subagent when its **description** matches the task

Generate the Project Spec

Inside the interactive REPL (normal mode), invoke the specs subagent:

```
Write a technical specification for mychatbot.
It should cover:
- The CLI chatbot interface (multi-turn
  conversation,
  history, exit command)
- The Streamlit web chatbot (session state,
  streaming)
- The shared LangChain chain module (model config,
  system prompt injection, memory management)
- Environment variable requirements
- Error handling expectations
Save to docs/specs/mychatbot-spec.md
```

Claude detects this matches the “specs” subagent description and routes automatically. You can also invoke explicitly:

```
# Explicit subagent invocation:
Use the specs subagent to write a spec for
mychatbot.

# In non-interactive mode:
claude -p "Use the specs subagent to write a spec
for
mychatbot. Save to docs/specs/mychatbot-spec.md"
```

```
git add docs/
git commit -m "docs: add mychatbot spec"
```

Architecture Design

Plan mode: read-only exploration before any edits

Plan Mode: Design Before Coding

Plan mode restricts Claude to **read-only exploration** – no file edits, no bash execution:

```
# Enter plan mode (cycles through modes):
Shift+Tab # Cycles: normal -> plan ->
auto-accept
```

```
# Or via command:
/plan # Enter plan mode
/plan refactor the auth module # With description
```

In Plan mode, ask for the architecture:

```
Read docs/specs/mychatbot-spec.md, then design the
project structure for mychatbot:
- Package layout under src/mychatbot/
- Module responsibilities (chain.py, cli.py,
  app.py,
  config.py)
- Data flow between modules
- Dependency graph
- What should be tested vs. what should be mocked
Show me the plan before creating any files.
```

After plan review, three approval options:

- “Yes, clear context and auto-accept edits” (recommended: fresh context for implementation)
- “Yes, and manually approve edits” (review each change)
- “Yes, auto-accept edits” (run with preserved context)

Plan Mode: Extended Thinking for Hard Problems

For complex architectural decisions, combine Plan mode with **Extended Thinking**:

```
# Toggle extended thinking inside a session:
Alt+T # Toggle extended thinking on/off

# Or launch with thinking:
claude "Design the architecture for mychatbot"
# Then press Alt+T before sending complex questions
```

In Plan mode with extended thinking enabled:

```
Given docs/specs/mychatbot-spec.md, should we use:
1. LangChain ConversationBufferMemory (simple,
  risk of
  token bloat), or
2. LangChain ConversationSummaryMemory (summarises
  old
  turns, lower token use)?
```

Consider: multi-turn history size, streaming support, Groq API limits, testability. Think carefully and give a recommendation with trade-offs.

Pro Tip: Extended thinking is especially valuable in Plan mode because it can explore multiple approaches without consuming execution tokens on wasted code.

Scaffold the Project Structure

Exit plan mode (press Shift+Tab). In normal mode:

```
Create the mychatbot package scaffold based on the
architecture plan:
- src/mychatbot/__init__.py
- src/mychatbot/chain.py (stub: make_chain())
- src/mychatbot/cli.py (stub: main())
- src/mychatbot/app.py (stub: Streamlit
  skeleton)
- src/mychatbot/config.py (stub: Config
  dataclass)
- tests/__init__.py
- tests/test_chain.py (stub)
- pyproject.toml (project metadata +
  deps)
- README.md (one-line placeholder)
```

```
git add .
git commit -m "chore: scaffold mychatbot package"
```

Context management tip:

```
/cost # Check token usage so far
/compact # Summarise older context to free space
# Claude preserves key decisions when
compacting
```

Implementation

File references, auto-accept mode, and session management

Implement chain.py

In normal mode (use @file: to reference files precisely):

```
@file:src/mychatbot/chain.py
@file:docs/specs/mychatbot-spec.md
Implement chain.py:
- make_chain(model: str, system_prompt: str) ->
  Callable
- Use ChatGroq from langchain_groq
- Accept conversation history as list[dict]
  (role/content)
- Return a callable that takes history and returns
  str
- Read GROQ_API_KEY from env via Config
```

- Raise `EnvironmentError` if key is missing
- Include full `type` hints and Google-style docstrings
- Handle Groq API errors with informative messages

Reviewing Claude's choices:

```
# Ask Claude to explain its implementation
decisions:

Why did you use ChatPromptTemplate instead of
MessagesPlaceholder? Is there a better approach
for multi-turn history?
```

```
git add src/mychatbot/chain.py
git commit -m "feat: implement LangChain chain
factory"
```

Implement cli.py and app.py CLI implementation:

```
@file:src/mychatbot/cli.py
Implement the CLI chatbot:
- main() as entry point
- Use Rich: Console, Panel (welcome), Markdown
  (responses)
- [You] prompt styled, [mychatbot] in green
- Maintain conversation history as list[dict]
- Commands: /exit or /quit to end, /clear to reset
  history
- Ctrl+C gracefully exits with "Goodbye!" message
- Call make_chain() from mychatbot.chain
- Model/system prompt configurable via env vars
```

Streamlit implementation:

```
@file:src/mychatbot/app.py
Implement the Streamlit chatbot:
- st.set_page_config: title="mychatbot"
- st.session_state["messages"] for conversation
  history
- st.session_state["chain"] to cache the chain
  callable
- st.chat_message bubbles for conversation display
- Sidebar: model selector, system prompt text area
- Spinner while waiting for the model response
- Handle missing GROQ_API_KEY with st.error and
  st.stop()
```

Auto-Accept Mode and Session Management

For bulk implementation work, use `auto-accept mode`:

```
# Switch to auto-accept (Claude applies all edits
  without
  # asking for approval per-file):
Shift+Tab # cycles to auto-accept mode

# Claude Code shows a running diff and cost in the
  status bar.
# You can interrupt at any time with Ctrl+C.
```

Session management:

```
# Resume most recent session after closing
  terminal:
claude -c

# Name the current session for later:
/rename mychatbot-implementation

# Resume a named session:
claude --resume mychatbot-implementation

# Fork a session to try an alternative approach:
claude --resume mychatbot-implementation
--fork-session
```

Run the chatbot to verify:

```
python -m mychatbot.cli # CLI interface
streamlit run src/mychatbot/app.py #
http://localhost:8501
```

Testing with Synthetic Data

Custom commands + skills for test generation

Create a Testing Skill

Skills are SKILL.md folders that Claude loads on-demand when the task context matches. Unlike CLAUDE.md (always loaded), skills are **lazy-loaded** – they don't bloat context until needed.

```
mkdir -p .claude/skills/pytest-patterns

cat > .claude/skills/pytest-patterns/SKILL.md <<
'EOF'
---
name: pytest-patterns
description: >
  Best practices and patterns for writing pytest
  tests
  for this project. Use when generating or
  reviewing
```

```
Python unit tests for mychatbot.
---
## Pytest Patterns for mychatbot

### Fixtures
- Use '@pytest.fixture' for shared setup
- 'mock_chain': returns a MagicMock callable
- 'sample_messages': list of user/assistant dicts

### Mocking
- Mock 'mychatbot.chain.ChatGroq': avoid real API
  calls
- Use 'pytest.raises' for exception testing
- Use 'monkeypatch' for env var testing

### Synthetic Data (Faker)
- Import 'faker.Faker' for realistic fake data
- Generate fake user messages, conversation
  histories
- Use '@pytest.mark.parametrize' for data-driven
  tests

### Coverage Target: 85%+
EOF
```

Create the /test-gen Custom Command

Custom commands live in `.claude/commands/` (project) or `~/.claude/commands/` (global):

```
mkdir -p .claude/commands

cat > .claude/commands/test-gen.md << 'EOF'
---
description: >
  Generate comprehensive pytest tests for a source
  file.
  Loads the pytest-patterns skill. Pass the source
  file
  path as the argument.
allowed-tools: Read, Edit, Write, Bash
---
Generate comprehensive pytest tests for $ARGUMENTS.

Before writing tests:
1. Load the pytest-patterns skill for this project
2. Read the source file to understand all
  functions/classes
3. Read the existing test file (if any) to avoid
  duplicates

Write tests covering:
- Happy path for each public function
- Edge cases (empty input, None, boundary values)
```

```
- Error cases (missing env vars, API failures --
  mock LLM)
- Use Faker to generate synthetic conversation data
  for multi-turn history tests

Output: complete updated test file, no
  placeholders.
EOF
```

Generate Tests with Synthetic Data

In the REPL:

```
/test-gen src/mychatbot/chain.py
```

Claude will generate something like:

```
# tests/test_chain.py
import pytest
from unittest.mock import patch, MagicMock
from faker import Faker
from mychatbot.chain import make_chain

fake = Faker()

@pytest.fixture
def mock_chat_groq():
    with patch("mychatbot.chain.ChatGroq") as mock:
        instance = mock.return_value
        instance.invoke.return_value.content =
            "Hello!"
        yield mock

@pytest.fixture
def sample_conversation():
    """Synthetic multi-turn conversation using
    Faker."""
    return [
        {"role": "user", "content":
            fake.sentence()},
        {"role": "assistant", "content":
            fake.paragraph()},
        {"role": "user", "content":
            fake.sentence()},
    ]

def
    test_make_chain_returns_callable(mock_chat_groq,
                                     monkeypatch):
    monkeypatch.setenv("GROQ_API_KEY", "test-key")
    chain = make_chain("llama-3.3-70b-versatile")
    assert callable(chain)
```

Generate Tests for cli.py and Run Suite

CLI tests will cover:

- /exit command terminates the loop
- /clear resets the conversation history
- Missing GROQ_API_KEY raises EnvironmentError
- KeyboardInterrupt (Ctrl+C) exits gracefully
- Rich console output contains expected text

```
/test-gen src/mychatbot/cli.py

# Run the full test suite:
pytest tests/ -v --tb=short

# With coverage:
pytest tests/ --cov=src/mychatbot
--cov-report=term-missing
# Target: >85% coverage

# If tests fail, describe the error:
The test test_make_chain_missing_key is failing
with:
KeyError: 'GROQ_API_KEY'. Here is the traceback:
[paste]
# Claude reads test + source and proposes the fix.

git add tests/
git commit -m "test: add unit tests with synthetic
  data"
```

Bonus: /test-run command

```
# .claude/commands/test-run.md runs full suite +
  coverage
# and summarises pass/fail count and low-coverage
  modules.
/test-run
```

Debugging

Debug subagent + context tools

Create a Debug Subagent

A Debug subagent has read + bash only – it investigates, never modifies:

```
cat > .claude/agents/debug.md << 'EOF'
---
name: debug
description: >
  Investigates bugs and runtime errors. Read-only:
  never
```

```
modifies files. Uses bash to run tests and
  inspect
  state. Outputs a structured findings report. Use
  when
  diagnosing a crash, test failure, or unexpected
  output.
model: claude-sonnet-4-6
allowed-tools: Read, Bash, Glob, Grep
---
You are a meticulous debugger. When given an error:
1. Read relevant source files
2. Trace the error from the stack trace
3. Run targeted bash commands to reproduce the
  issue
4. Identify the root cause
5. Suggest a minimal fix (do NOT apply it: report
  only)

Output format:
## Root Cause
## Affected Files
## Minimal Reproduction
## Suggested Fix
EOF
```

Debug Workflow Demo

Scenario: The CLI crashes when the user types a very long message.

Step 1 – Invoke debug subagent:

```
Use the debug subagent. The CLI chatbot crashes
  when the
  user enters a message longer than ~4000 characters.
Error:
groq.BadRequestError: Error code: 400
  Request too large for model
  llama-3.3-70b-versatile

Stack trace:
File "src/mychatbot/chain.py", line 34, in
  __call__
  response = self.llm.invoke(messages)
```

Step 2 – Apply fix in normal mode:

```
@file:src/mychatbot/chain.py
Add input validation: if total character count of
  all
  messages exceeds 12000 chars, truncate the oldest
  messages (not the system message) until it fits.
Warn via the logging module when truncation occurs.
```

Step 3 – Visualise context usage:


```
/context    # Shows token usage breakdown
/cost       # Shows session cost so far
Ctrl+O     # Toggle verbose mode (shows hook
            stdout/stderr)
```

Refactoring

Using skills and rules for consistent improvements

Create a Refactoring Skill

```
mkdir -p .claude/skills/refactor-patterns

cat > .claude/skills/refactor-patterns/SKILL.md <<
'EOF'
---
name: refactor-patterns
description: >
  Patterns and criteria for safe refactoring of
  Python
  code. Use when asked to refactor or improve code
  quality in the mychatbot codebase.
---
## Refactoring Rules

### Safety First
- Never change public API signatures without
  updating
  all callers and the spec doc
- Run tests before and after: confirm no regression

### Python Improvements to Look For
- Replace dict literals with TypedDict or dataclass
- Extract magic strings (model names, env var
  names)
  into constants at module level
- Replace print() with logging module
- Move configuration into a Config dataclass

### Code Smell Checklist
- Functions longer than 40 lines: consider
  splitting
- Duplicate code across cli.py and app.py: extract
- Nested if/else depth > 3: refactor to early
  returns
EOF
```

Refactor: Extract Shared Config

In normal mode:

Load the refactor-patterns skill, then:

1. Identify duplicated configuration logic between src/mychatbot/cli.py and src/mychatbot/app.py (both read mychatbot_MODEL and mychatbot_SYSTEM_PROMPT from env vars)
2. Create src/mychatbot/config.py with a Config dataclass that reads all env vars in one place
3. Update cli.py and app.py to import and use Config
4. Add a test in tests/test_config.py covering defaults and env var overrides

Use @file: to read each file before editing.
Show me the plan before making any changes.

Review the plan, then approve:

```
Looks good. Go ahead and implement it.

# Run tests after refactor:
pytest tests/ -v

git add .
git commit -m "refactor: extract Config dataclass"
```

Refactor: Add Structured Logging

```
Replace all print() debug statements in
src/mychatbot/
with Python's logging module:
- Create logger = logging.getLogger(__name__) at
  the top of each module
- Use logging.INFO for user-facing info
- Use logging.DEBUG for internal state
- Use logging.ERROR for caught exceptions
- In cli.py: configure basicConfig with level from
  env var LOG_LEVEL (default: WARNING)
- Do NOT change any Rich console output: that stays
- Only replace bare print() debug statements

# The CLAUDE.md rule "Never catch bare except"
# is respected automatically -- no need to repeat
it.

pytest tests/ -v

git add .
git commit -m "refactor: replace print with
logging"
```

Rules files add another layer:

```
# .claude/rules/python-style.md already says
# "Use Black formatting" -- Claude applies it
# automatically on every file it touches.
```

Documentation

Docs subagent + auto-generated README

Create a Docs Subagent

```
cat > .claude/agents/docs.md << 'EOF'
---
name: docs
description: >
  Documentation writer. Generates README, API docs,
  usage guides, and changelog entries. Writes
  Markdown
  files but cannot modify Python source. Use when
  generating or updating documentation for any
  module.
model: claude-sonnet-4-6
allowed-tools: Read, Write, Bash
---
You are a technical documentation expert. When
generating:
- Read all relevant source files first
- Use docs/specs/ as the source of truth for
  behaviour
- Write clear, copy-pasteable code examples
- Include a "Quick Start" section in every README
- Use GitHub-flavoured Markdown
- Never invent features not present in the source
  code
EOF
```

Generate README and API docs:

```
Use the docs subagent to generate a complete
README.md
for mychatbot: badges, description, features,
installation,
quick start (CLI + Streamlit), env var table,
development
guide. Then generate docs/api.md covering the
public API
of chain.py and config.py with usage examples.

git add README.md docs/
git commit -m "docs: generate README and API docs"
```

Generate CHANGELOG and /changelog Command

```
# Generate CHANGELOG via docs subagent:
Use the docs subagent to generate a CHANGELOG.md in
Keep-a-ChangeLog format. Run 'git log --oneline'
to get
the commit history, then group into:
Added (feat:), Fixed (fix:), Changed (refactor:),
Documentation (docs:). Tag as version 0.1.0 with
today's
date.
```

Create a reusable /changelog command:

```
cat > .claude/commands/changelog.md << 'EOF'
---
description: >
  Update CHANGELOG from recent git commits.
  Pass a time range or tag as the argument.
allowed-tools: Read, Write, Bash
---
Run 'git log --oneline --since="$ARGUMENTS"'.
Update CHANGELOG.md with a new version section
grouping
commits by type: feat->Added, fix->Fixed,
refactor->Changed, docs->Docs.
Suggest a semantic version bump based on the
changes.
EOF

# Usage:
/changelog "1 week ago"
/changelog "last tag"
```

Code Review

Review subagent + permission system

Create a Review Subagent with Strict Permissions

```
cat > .claude/agents/review.md << 'EOF'
---
name: review
description: >
  Code reviewer. Performs thorough code review
  covering
  correctness, security, performance, style, and
  test
  coverage. Read-only: cannot modify any files.
  Use when
```

```
  reviewing a file or the entire codebase before a
  PR.
model: claude-opus-4-6
allowed-tools: Read, Grep, Glob
---
Perform a code review. For each file, check:
1. Correctness: logic bugs, off-by-one, type errors
2. Security: hardcoded secrets, injection,
  unvalidated input
3. Performance: unnecessary loops, blocking calls
4. Style: matches CLAUDE.md conventions
5. Test coverage: untested branches or edge cases
Output: structured report with severity
(Critical / Major / Minor) and suggested fix.
EOF
```

Key difference: allowed-tools in subagent definitions replaces OpenCode's explicit permission: deny blocks. Claude Code's permission system works hierarchically: project settings.json → user settings.json → subagent allowed-tools.

Run a Full Code Review

```
Use the review subagent to review the entire
src/mychatbot
codebase. Focus on:
1. Security: are API keys ever logged or printed?
2. Error handling: are all Groq API errors caught?
3. Streamlit: any session state race conditions?
4. Tests: what branches are not covered?
5. Type correctness: any missing type hints?

For each issue note: file and line, severity
(Critical/Major/Minor), what the issue is, how to
fix it.
```

Apply findings:

```
Apply all Critical and Major fixes identified in
the
review report. For each fix, show what changed.
```

Create a reusable /review command:

```
cat > .claude/commands/review.md << 'EOF'
---
description: >
  Full code review of $ARGUMENTS (or workspace)
allowed-tools: Read, Grep, Glob
---
Use the review subagent. Review $ARGUMENTS for
correctness, security, performance, and style.
Group findings by severity.
EOF
```

Permission System: Fine-Grained Control

Configure permissions in .claude/settings.json to protect your project:

```
{
  "permissions": {
    "allow": [
      "Read",
      "Bash(npm run:*)",
      "Bash(git commit:*)",
      "Bash(git add:*)",
      "Edit(src/**)",
      "Write(src/**)"
    ],
    "deny": [
      "Read(.env*)",
      "Read(secrets/**)",
      "Bash(rm -rf:*)",
      "Bash(sudo:*)",
      "Edit(package-lock.json)"
    ],
    "ask": [
      "WebFetch",
      "Bash(curl:*)",
      "Bash(docker:*)"
    ],
    "defaultMode": "acceptEdits"
  }
}
```

Scopes: allow (auto-proceed), deny (always block), ask (prompt user). Enterprise settings at /etc/claude-code/managed-settings.json cannot be bypassed.

GitHub Workflows via MCP

Connecting Claude Code to external services

Configure the GitHub MCP Server

MCP (Model Context Protocol) connects Claude Code to external tools. Claude Code has a first-class claude mcp CLI:

```
# Add GitHub MCP server (scoped to this project):
claude mcp add github \
  --transport http \
  https://api.githubcopilot.com/mcp \
  -H "Authorization: Bearer $GITHUB_TOKEN"

# Or use the official npm server (stdio):
claude mcp add-json github \
  '{"command": "npx",

  "args": ["-y", "@modelcontextprotocol/server-github"]},
```

```
"env":{"GITHUB_TOKEN":"${GITHUB_TOKEN}"}'}

# Scope flags:
# --scope local (default): only you, current
#   project
# --scope project: shared via .mcp.json
#   (committed)
# --scope user: all your projects

# List configured servers:
claude mcp list

# Create a shared .mcp.json for the team:
claude mcp add --scope project github ...
```

GitHub MCP Workflows

With the GitHub MCP server active, Claude Code can manage your repo:

Create issues from code review findings:

```
Create GitHub issues for each Critical and Major
finding
from the code review. Title each clearly, add
labels:
"bug" for Critical, "enhancement" for Major.
Assign them all to me.
```

Create a feature branch and PR:

```
Create a feature branch "feat/add-streaming",
implement streaming responses in chain.py using
LangChain's streaming callback. When done, create a
GitHub PR with title "feat: add streaming
responses",
body describing changes, testing done, and a link
to the relevant issue.
```

Non-interactive mode for CI (GitHub Actions):

```
# Read latest unresolved PR comment and respond:
claude -p "Read the latest unresolved PR comment on
PR #1 and either apply the change or post a
response
explaining why we should not."
```

Add More MCP Servers

Add additional MCP servers for common workflows:

```
# Sequential Thinking (no API key needed):
claude mcp add-json sequential-thinking \
'{"type":"stdio","command":"npx",
"args":["-y",
```

```
"@modelcontextprotocol/server-sequential-thinking"]}'

# Brave Search:
claude mcp add-json brave-search \
'{"type":"stdio","command":"npx",
"args":["-y",
"@modelcontextprotocol/server-brave-search"],
"env":{"BRAVE_API_KEY":"${BRAVE_API_KEY}"}'}

# PostgreSQL (for database queries):
claude mcp add-json postgres \

'{"command":"npx","args":["-y","@benborla29/mcp-server-
"postgresql://user:pass@localhost/mydb"]}'
```

Use MCP tools in session:

```
# MCP tools appear as slash commands:
/mcp_github_create-issue
/mcp_playwright_create-test

# Or ask naturally:
Search for the latest LangChain 0.3 changelog and
tell me if any of our chain.py code needs updating.
```

Hooks: Deterministic Automation

A Claude Code-exclusive feature: enforce rules that always run

The Hooks System: Overview

Hooks are shell commands (or prompt/agent calls) that fire automatically at lifecycle events. Unlike prompts, **hooks are deterministic** – they always run regardless of model behaviour.

12 lifecycle events:

Event	When it fires
PreToolUse	Before any tool call (can block)
PostToolUse	After a tool completes
Stop	When Claude finishes responding
SubagentStop	When a subagent finishes
SessionStart	At the start of a new session
SessionEnd	When a session closes
UserPromptSubmit	When user submits a message
Notification	When Claude needs attention
PreCompact	Before context compaction

Three handler types:

- **command**: run a shell script (fast, deterministic)
- **prompt**: send to a Claude model for AI judgement

- **agent**: spawn a subagent with tool access for deep checks

Exit codes: 0 = proceed, 2 = block + send reason to Claude, other = proceed + log stderr.

Hooks: Auto-Format and Block Dangerous Commands

Add hooks to .claude/settings.json:

Auto-format Python after every file edit (PostToolUse):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "black \"${CLAUDE_FILE_PATHS}\"
2>/dev/null || true"
        }]
      }
    ]
  }
}
```

Block dangerous bash commands (PreToolUse):

```
"PreToolUse": [
  {
    "matcher": "Bash",
    "hooks": [{
      "type": "command",
      "command": "INPUT=$(cat); CMD=$(echo
$INPUT | jq -r '.tool_input.command'); echo
\"$CMD\" | grep -qE 'rm -rf|DROP TABLE' &&
echo 'Blocked: destructive command' >&2 &&
exit 2 || exit 0"
    }]
  }
]
```

Hooks: Run Tests Before Stopping and Desktop Alerts

Auto-run tests when Claude finishes (Stop hook):

```
"Stop": [
  {
    "hooks": [{
      "type": "command",
      "command": "INPUT=$(cat); [ \"$(echo
$INPUT | jq -r '.stop_hook_active')\" =
'true' ] && exit 0; cd
```



```
\"$CLAUDE_PROJECT_ROOT\" && pytest tests/ -q
--tb=short || exit 2"
  }}
}
]
```

Desktop notifications when Claude needs you (Notification hook):

```
"Notification": [
{
  "hooks": [{
    "type": "command",
    "command": "osascript -e 'display
notification \"Claude needs attention\" with
title \"Claude Code\"' 2>/dev/null ||
notify-send 'Claude Code' 'Needs attention'"
  }]
}
]
```

Load git context at session start (SessionStart hook):

```
"SessionStart": [
{
  "hooks": [{
    "type": "command",
    "command": "git status; git log
--oneline -5"
  }]
}
]
```

Hooks: AI-Powered Security Check (Prompt Hook)

For decisions requiring judgement, use **prompt hooks**:

```
"PreToolUse": [
{
  "matcher": "Write|Edit",
  "hooks": [{
    "type": "prompt",
    "prompt": "Validate file write safety.
Check: system paths, credentials, path
traversal, sensitive content. Input:
$TOOL_INPUT. Return 'approve' or 'deny'.",
    "timeout": 30
  }]
}
]
```

Deep post-edit analysis (agent hook):

```
"PostToolUse": [
{
  "matcher": "Edit",
  "hooks": [{
    "type": "agent",
    "description": "Check for syntax errors,
security vulnerabilities, and breaking
changes after this file edit.",
    "allowed-tools": ["Read", "Bash(python
-m py_compile:*)"]
  }]
}
]
```

Debugging hooks: Press Ctrl+0 to toggle verbose mode and see hook stdout/stderr in the transcript. Run `/hooks` to list all active hooks.

Plugins: Packaged Workflows

Another Claude Code-exclusive feature: bundle and share everything

Plugins: Overview and Structure

Plugins bundle skills, subagents, hooks, commands, and MCP configs into a single installable unit. They solve the “works-on-my-machine” problem for team tooling.

```
# Plugin directory structure:
mychatbot-plugin/
.claude-plugin/
  plugin.json           # Metadata (name,
                        # version, author)
commands/
  test-gen.md           # /test-gen command
  review.md             # /review command
agents/
  debug.md              # debug subagent
  review.md             # review subagent
skills/
  pytest-patterns/
    SKILL.md
  refactor-patterns/
    SKILL.md
hooks/
  hooks.json            # PostToolUse formatter,
                        # etc.
.mcp.json               # MCP server configs
scripts/
  format-code.sh        # Hook scripts
```

plugin.json minimal example:

```
{
  "name": "mychatbot-dev-tools",
  "version": "1.0.0",
  "description": "Dev workflow for mychatbot
project",
  "author": { "name": "Your Team" }
}
```

Installing and Using Plugins

```
# Install a local plugin:
claude plugin add --path ./mychatbot-plugin

# Install from marketplace:
claude plugin install <plugin-name>

# List installed plugins:
claude plugin list

# Use namespaced skills (avoids collisions):
/mychatbot-dev-tools:review-component

# Trust a repo folder so plugins auto-install for
team:
# Add to .claude/settings.json:
{
  "plugins": {
    "autoInstall": [
      { "path": "./mychatbot-plugin" }
    ]
  }
}
```

Package our demo config as a plugin:

```
# Move our .claude/ config into a plugin structure:
mkdir -p mychatbot-plugin/.claude-plugin
echo
'{"name":"mychatbot-dev-tools","version":"1.0.0"}'
\
> mychatbot-plugin/.claude-plugin/plugin.json
cp -r .claude/agents mychatbot-plugin/agents/
cp -r .claude/skills mychatbot-plugin/skills/
cp -r .claude/commands mychatbot-plugin/commands/
# Now any team member can: claude plugin add
--path ./mychatbot-plugin
```

Bonus: CI/CD and Advanced Techniques[0.5em] Non-interactive mode, parallel subagents, IDE integration

Non-Interactive Mode in CI/CD

Claude Code's `claude -p` (print mode) enables AI-powered CI workflows:

GitHub Actions example (`.github/workflows/ai-review.yml`):

```
name: AI Code Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - run: curl -fsSL https://claude.ai/install.sh | bash
      - name: Run AI review on changed files
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          CHANGED=$(git diff --name-only origin/main HEAD \
            | grep '\.py$' | tr '\n' ' ')
          claude -p "Use the review subagent.
Review
these changed files: $CHANGED.
Post findings as a GitHub PR comment
using the github MCP server."
```

Parallel Subagents and PR-Linked Sessions

Parallel subagents (up to 10) let Claude explore concurrently:

```
# Claude Code can spawn multiple subagents in parallel:
# (orchestrated via a custom command or inline prompt)

cat > .claude/commands/parallel-review.md << 'EOF'
---
description: >
  Run a security review and a performance review in parallel, then merge findings into a report.
---
Spawn two subagents in parallel:
1. Use the review subagent with focus ONLY on security
   (secrets, injection, input validation)
2. Use the review subagent with focus ONLY on performance
   (N+1 queries, blocking calls, memory leaks)
```

```
Wait for both to finish, then write a merged report to
docs/review-$(date +%Y%m%d).md
EOF

/parallel-review
```

PR-linked sessions (unique to Claude Code):

```
# Start a session linked to a specific PR:
claude --from-pr 42          # By PR number
claude --from-pr https://github.com/org/repo/pull/42

# Sessions auto-link when you create a PR via gh CLI
# during a session. Resume later:
claude --resume              # Shows PR-linked sessions
```

IDE Integration

Claude Code integrates into VS Code and JetBrains IDEs. All CLAUDE.md, settings, and MCP servers are shared:

VS Code extension:

```
# Install from VS Code Marketplace:
# Search: "Claude Code"

# Or via terminal:
code --install-extension anthropic.claude-code

# Use inside VS Code:
# Ctrl+Shift+P -> "Claude Code: Open"
# Or: click the Claude icon in the sidebar
```

JetBrains plugin:

```
# Install from JetBrains Marketplace:
# Settings -> Plugins -> Search "Claude Code"
# Supports: IntelliJ IDEA, PyCharm, WebStorm, etc.
```

What IDE integration adds:

- Inline diff view as Claude edits files
- One-click approve/reject per file change
- Editor context automatically sent (open file, selection)
- Same session continues from terminal – no context loss

Image and Multimodal Input

Claude Code accepts images as input – paste screenshots, UI mockups, or diagrams directly:

```
# Paste a screenshot into the REPL (macOS: Cmd+V):
# Claude sees the image and can act on it.

# Example workflows:

# 1. Screenshot of a bug:
# [paste screenshot of browser error]
"This is the error. Find the cause in src/mychatbot/app.py"

# 2. UI mockup -> Streamlit code:
# [paste UI wireframe image]
"Implement this layout in src/mychatbot/app.py using
st.columns, st.container, and the sidebar design shown"

# 3. Architecture diagram -> scaffold:
# [paste architecture diagram]
"Create the file structure shown in this diagram.
Add stub implementations for each module."

# 4. Test failure screenshot:
# [paste pytest output screenshot]
"Fix the failing tests shown in this screenshot"
```

Pro Tip: Drag-and-drop images directly into the terminal on supported platforms (iTerm2, Windows Terminal).

Workshop Summary

What We Built

Project mychatbot/: complete from scratch:

- `src/mychatbot/chain.py`: LangChain + Groq chain factory
- `src/mychatbot/cli.py`: Rich CLI chatbot with history
- `src/mychatbot/app.py`: Streamlit multi-turn chatbot
- `src/mychatbot/config.py`: Centralized env-based config
- `tests/`: Full pytest suite with synthetic Faker data
- `docs/specs/mychatbot-spec.md`: Technical specification
- `README.md` + `docs/api.md`: Auto-generated docs
- `CHANGELOG.md`: Conventional commit-based changelog

Claude Code config committed to repo:

- `CLAUDE.md`: Project intelligence file
- `.claude/agents/`: specs, debug, docs, review sub-agents

- `.claude/skills/`: `pytest-patterns`, `refactor-patterns`
- `.claude/commands/`: `/test-gen`, `/test-run`, `/review`, `/docs`, `/changelog`
- `.claude/settings.json`: Permissions, hooks, MCP configs
- `.mcp.json`: Shared MCP server definitions (team)
- `mychatbot-plugin/`: Packaged plugin for team distribution

Claude Code SDLC Cheat Sheet

Task	Claude Code Command/Technique
Bootstrap project	<code>/init</code> → <code>CLAUDE.md</code>
Write spec	<code>specs</code> subagent
Design architecture	<code>Plan</code> mode (<code>Shift+Tab</code>)
Hard problem	<code>Plan</code> mode + <code>Alt+T</code> (thinking)
Write code	Normal mode + <code>@file</code> :
Bulk editing	<code>Auto-accept</code> mode (<code>Shift+Tab</code> x2)
Generate tests	<code>/test-gen <file></code> (custom cmd)
Run test suite	<code>/test-run</code> (custom cmd)
Debug an error	<code>debug</code> subagent
Context management	<code>/compact</code> , <code>/cost</code>
Refactor safely	<code>refactor-patterns</code> skill
Generate README	<code>docs</code> subagent
Update CHANGELOG	<code>/changelog "last week"</code>
Code review	<code>review</code> subagent
Auto-format on save	<code>PostToolUse</code> hook
Block bad commands	<code>PreToolUse</code> hook
Run tests on stop	<code>Stop</code> hook
Create GitHub issue	<code>GitHub MCP</code>
Create PR	<code>GitHub MCP</code>
Resume session	<code>claude -c</code> or <code>claude --resume</code>
Share config	<code>Plugin (claude plugin add)</code>

Key Lessons from the Workshop

- **CLAUDE.md is your multiplier.** Write it carefully and every session starts smarter. Nested `CLAUDE.md` files in subdirectories add directory-specific context. Commit them all to `Git`.

- **Plan before Build.** Press `Shift+Tab` to enter `Plan` mode for anything non-trivial. Clear context on approval to give the plan a fresh window. Avoid costly rewrites.
- **Hooks guarantee; prompts suggest.** Use hooks for anything that *must* always happen: formatting, security checks, test runs. Never rely on the model to remember.
- **Route by model tier.** Use `Haiku` subagents for exploration, `Sonnet` for daily coding, `Opus` for architecture and security review. Save cost without sacrificing quality.
- **Skills keep context lean.** Lazy-loaded skills don't consume tokens until needed. Don't put everything in `CLAUDE.md`; use skills for domain knowledge and patterns.
- **MCP extends reach.** `GitHub`, databases, search – connect once and the agent acts across your entire stack. Use `--scope project` to share configs with the team.
- **Plugins solve team distribution.** Package your entire workflow once; the team installs it with one command. Hooks, skills, agents, and commands all travel together.
- **Plan mode is free exploration.** No risky edits, no wasted tokens on bad approaches. Use it liberally on unfamiliar codebases and complex features.

Exercises for Self-Study

- **Exercise 1:** Add a `/exit` history command to `cli.py` that saves conversation to `./mychatbot.history.json`. Use `/test-gen` to cover it.
- **Exercise 2:** Create a “Security” subagent that only checks for API key leaks, SQL injection patterns, and unvalidated inputs. Test it on an intentionally buggy snippet.

- **Exercise 3:** Add a PostgreSQL MCP server and a `/db-query` custom command. Have the chatbot store conversation logs in a database.
- **Exercise 4:** Write a `PreToolUse` hook using a **prompt handler** that checks whether any `Edit` to `src/` has a corresponding test file. Block the edit if there is no test.
- **Exercise 5:** Set up the GitHub Actions workflow for AI code review on your own repository. Trigger a PR and observe the auto-review in the PR comments.
- **Exercise 6:** Use `claude -p` in non-interactive mode to auto-generate release notes from the last 10 commits and post them as a GitHub release via the MCP server.
- **Exercise 7:** Package the entire workshop config into a distributable plugin. Share it with a colleague via `Git`. They install it with one `claude plugin add` command.

Resources

- **Claude Code Docs:** code.claude.com/docs
- **Claude Code Overview:** docs.claude.com/en/docs/claude-code
- **Official GitHub:** github.com/anthropics/claude-code
- **Awesome Claude Code (community):** github.com/hesreallyhim/awesome-claude-code
- **Claude Code How-To Guide:** github.com/luongnv89/claude-code-how-to-guide
- **MCP Server Registry:** github.com/modelcontextprotocol/servers
- **LangChain + Groq:** python.langchain.com/docs/integrations/groq
- **Groq Console (API keys):** console.groq.com
- **Streamlit Docs:** docs.streamlit.io
- Blake Crosley, “Claude Code CLI: The Complete Guide”, blakecrosley.com, 2026.
- Muneeb Ahmad, “Claude Code Extensions Explained”, Medium, Mar 2026.

Workshop complete. You now have a production-ready Claude Code workflow.[0.5em] Happy coding!